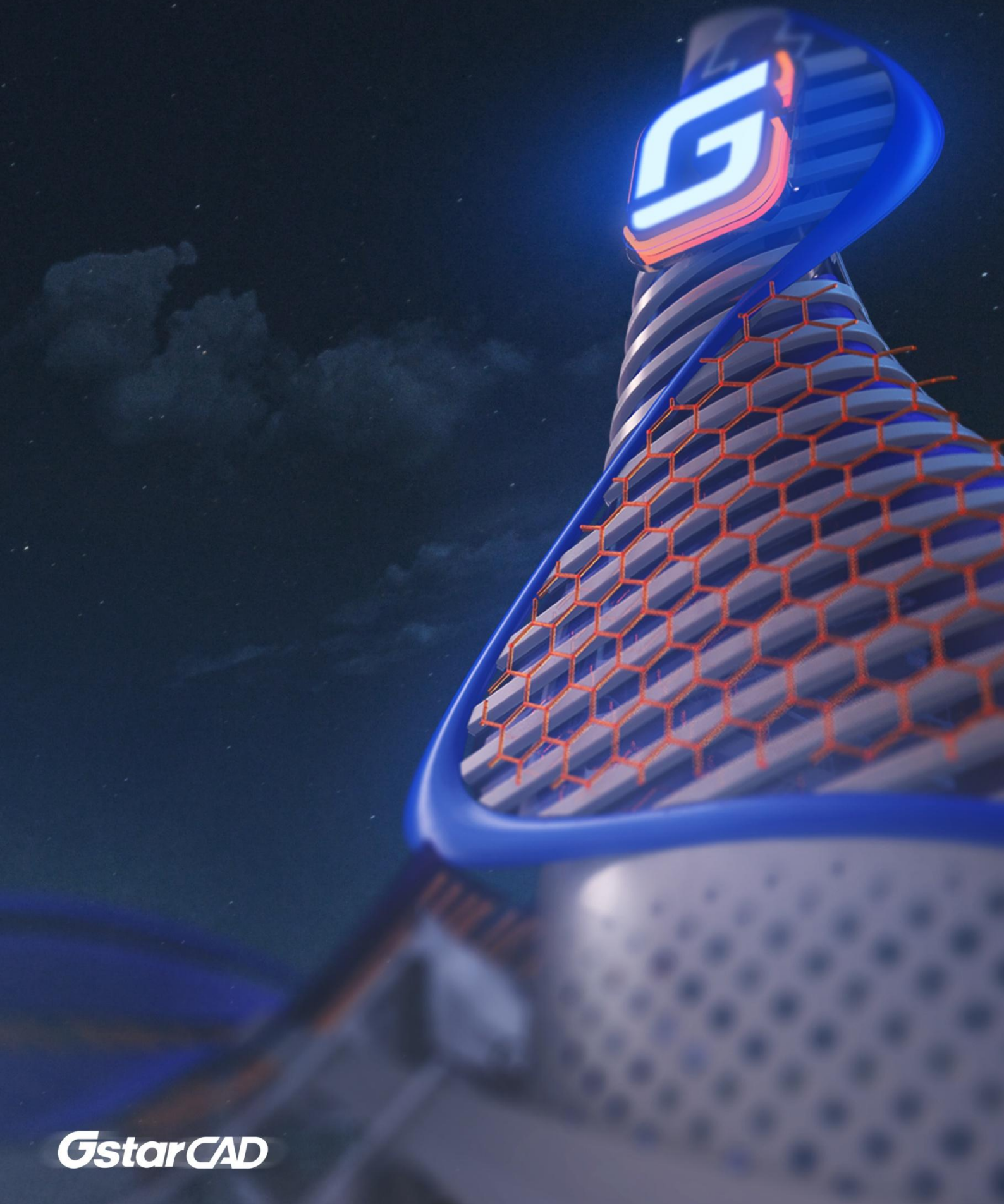
GRX Migration Guide

GstarCAD 2020



GstarCAD

Table of Contents

1. Brief Introduction 3
2. Development Environment 4
3. Original Code 5
4. Project Configuration 6
5. Transplanting ARX project with VS2017 8
   - 5.1. The Configuration of Target Extension 8
   - 5.2. The Configuration of Include Directories 8
   - 5.3. The Configuration of Preprocessor 9
   - 5.4. The Configuration of Linker 10
   - 5.5. Compile Program 11
6. The differences between GRX and ARX interface 12
   - 6.1. Function: monitorInputPoint 12
   - 6.2. Function: intersectWith 14
   - 6.3. Function: AcDbField::evaluate 14
   - 6.4. Interface for “Plot” 16
   - 6.5. Gs initialization 17
   - 6.6. Gs Pan 19
   - 6.7. acedGetArgs() release issue 19
7. Comparison of GRX and ARX libraries 20
   - 7.1. GRX class library description 20
     - 7.1.1. GcRx 20
     - 7.1.2. GcEd 20
     - 7.1.3. GcDb 21
     - 7.1.4. GcGi 21
     - 7.1.5. GcGe 21
8. Copyright 22

1. Brief Introduction

Application Developers from various industries have developed a large number of ARX applications base on AutoCAD®, and one of the highlights of GstarCADGRX is GstarCAD achieves source-level compatibility with AutoCAD®ARX programs, which means one set of code can be suitable for two platforms. Developers can migrate the ARX program which runs on the AutoCAD platform to GstarCAD with almost no code changing. Developer can refer this document to easy migrate the code to GstarCAD.

2. Development Environment

- Microsoft Visual Studio Enterprise 2017（Version 15.9.17）
- CPU：

Basic Requirement：1.6 GHz CUP，Recommend：3.0 GHz CPU and above

- RAM：
- Basic Requirement：2 GB，Recommend：8 GB and above
- Operation System
- Windows 10 version 1507 and advanced version：Including home version, professional version, education version and enterprise version（not support LTSC and Windows 10 S）
- Windows 8.1（with updated 2919355）： Core version, professional version and enterprise version
- Windows 7 SP1（with latest Windows updated）： Including home version, professional version, enterprise version and Ultimate version
- Monitor Resolution:
- Traditional monitor：1028 x 800 and above CRT monitor, High resolution and 4K (3840 x 2160) monitor
- GRX SDK 2020
- GstarCAD 2020
- .NET Framework 4.8 and above

3. Original Code

For any ObjectARX/hARX application under ObjectARX 2010 or a newer version, the application source code needs to be transplanted to VC10 SP1 build environment first, then it needs to be compiled in the form of multi-byte character sets and multi-thread, including non-ARX public module.

4. Project Configuration

1) Header file includs path：<sdkpath>\inc\Arx2010

2) Accessory library directory： <sdkpath>\lib-x86 或<sdkpath>\lib-x64

3) Extension of the object file：.grx

4) Precompile macro: _TOOLKIT_IN_DLL_, which has to be defined, otherwise, symbols could not be found to export as linking.

5) DLL Entry Function:

The Entry Point Function of a GRX program is gcrxEntryPoint. To be compatible with ARX, acrxEntryPoint still could be used as the Entry Point of a module but with grxport.lib been imported. This module is a static link library, providing an auto mapping from GRX Entry Point to ARX Entry Point. Correspondingly, RxExport.def will be set as the module definition file under path <sdkpath>\inc\Arx2010.

6) Accessory Items：

Lib Name	Be imported	Relative ARX	description
grxport.lib	always	No	acrxEntryPoint entry mapping module
TD_Alloc.lib	Usually	No	Memory Management Module
TD_Root.lib	Usually	AcRx Module	Basic running module
TD_DbRoot.lib	Usually	AcDb Module	Database base module
TD_Db.lib	Usually	AcDb Module	Data management module
TD_Ge.lib	Usually	AcGe Module	Universal Geometry Library
TD_Gi.lib	Usually	AcGi Module	Graphics drawing interface
TD_Gs.lib	Usually	AcGs Module	Graphics System Management Module
gcap.lib	Usually	AcAp Module	Document Management Module
gcui.lib	Usually	AcUi Module	MFC extension module
gcut.lib	Usually	AcUt Module	Auxiliary Interface
gced.lib	Usually	AcEd Module	User interaction module
gcdb.lib	Usually	AcDb Module	ADS database access interface

gcad.lib	Usually	ACAD Module	Application interface management
gcax.lib	occasionally	AcAx Module	COM base class interface
gcgs.lib	occasionally	AcGs Module	Graphics System
Others	Barely		Most are internal us

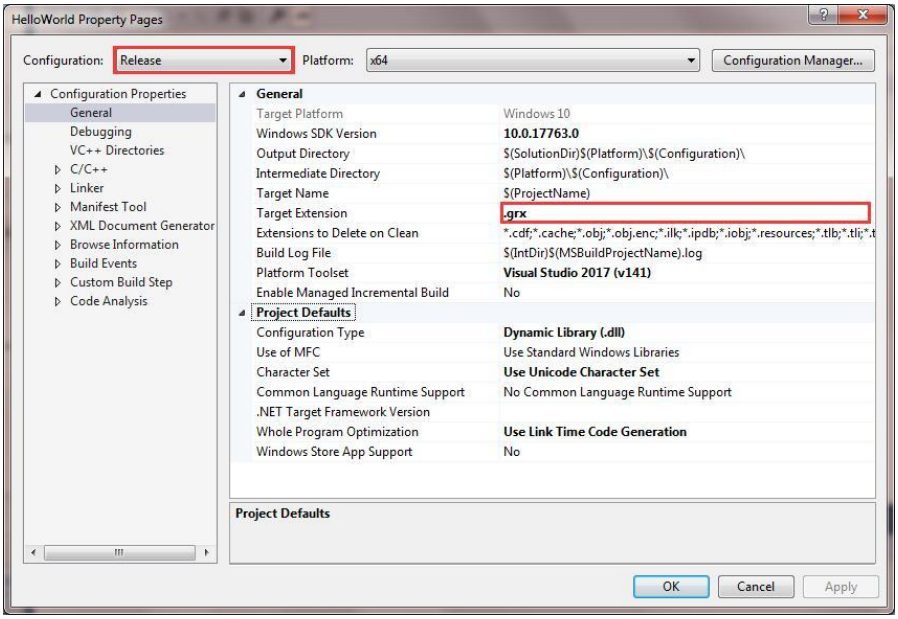
NOTE: <sdkpath> indicates the installation path of GstarCAD SDK, for example: *C:\grxsdk*.

grxport.lib must be placed in the first position of additional dependencies.

5. Migrate ARX project with VS2017

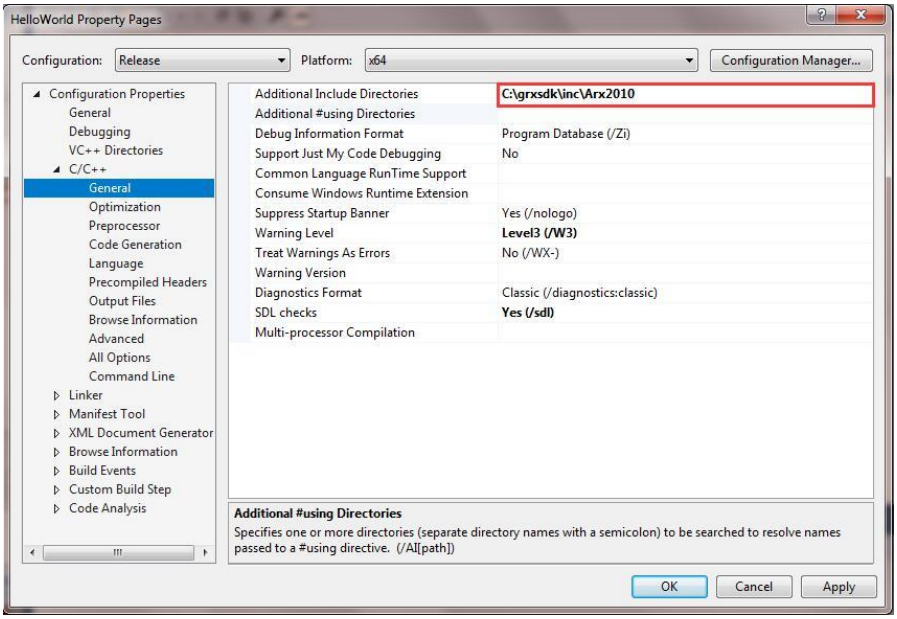
5.1. The Configuration of Target Extension

Select the General Node under Configuration Properties, and set Target Extension as .GRX. Meanwhile set Configuration as Release.



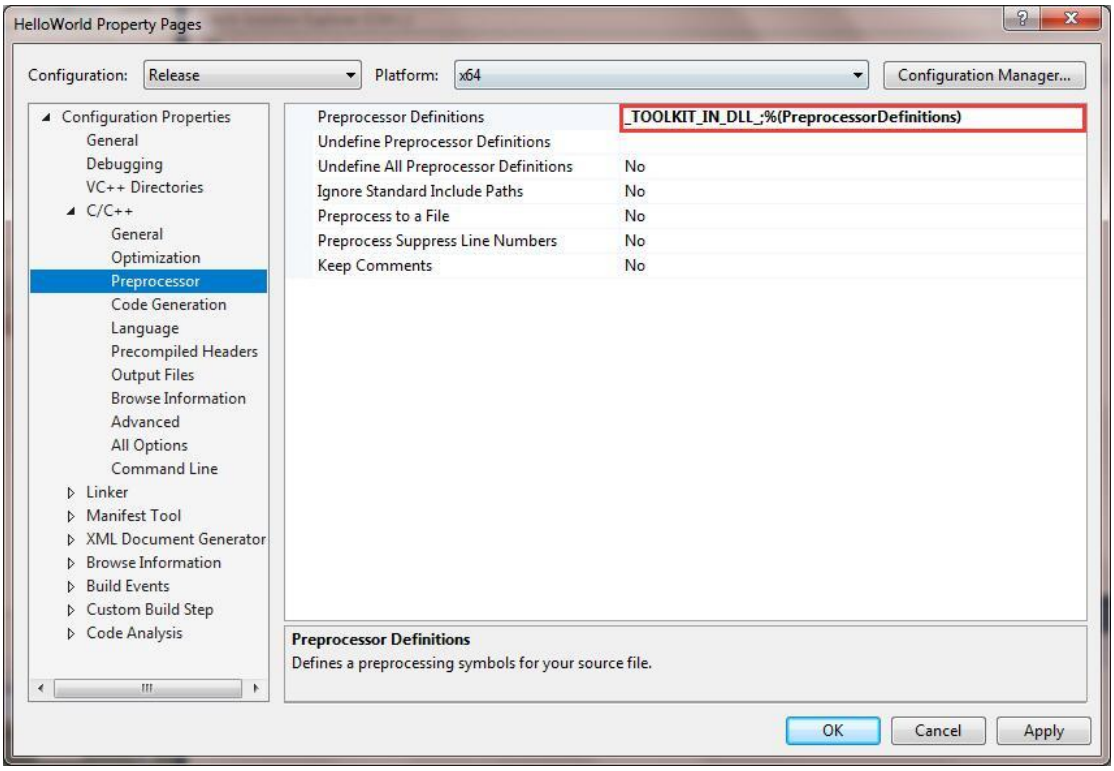
5.2. The Configuration of Include Directories

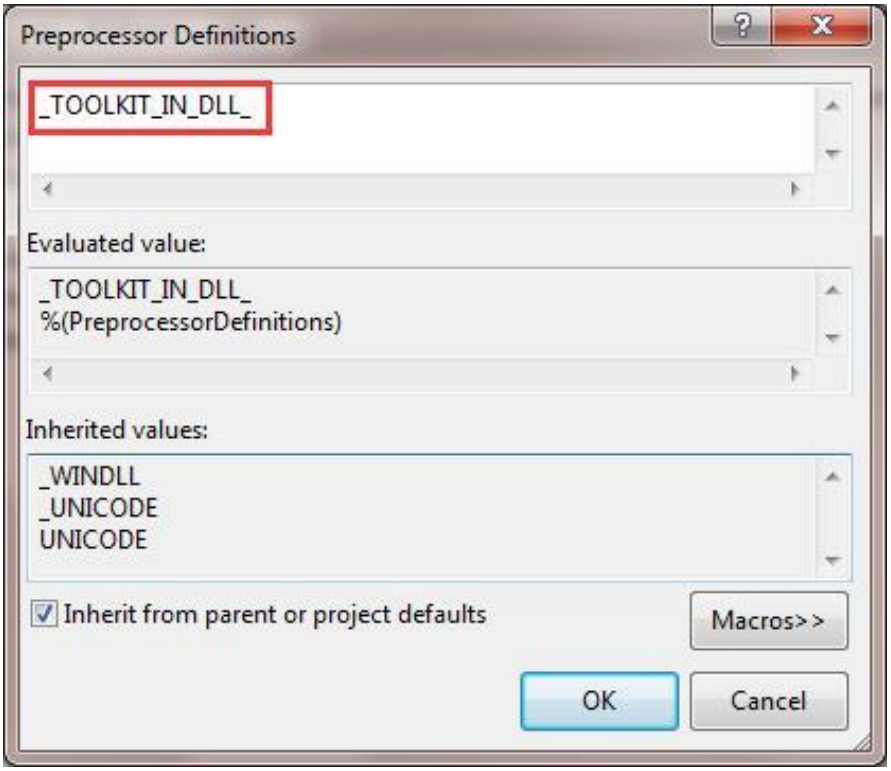
Select the C/C++ Node under Configuration Properties and set Additional Include Directories of General as <sdkpath>\inc\Arx2010 as shown below.



5.3. The Configuration of Preprocessor

Select the Preprocessor under C/C++ from Configuration Properties. Add Marco _TOOLKIT_IN_DLL_ into Preprocessor Definitions as shown below.





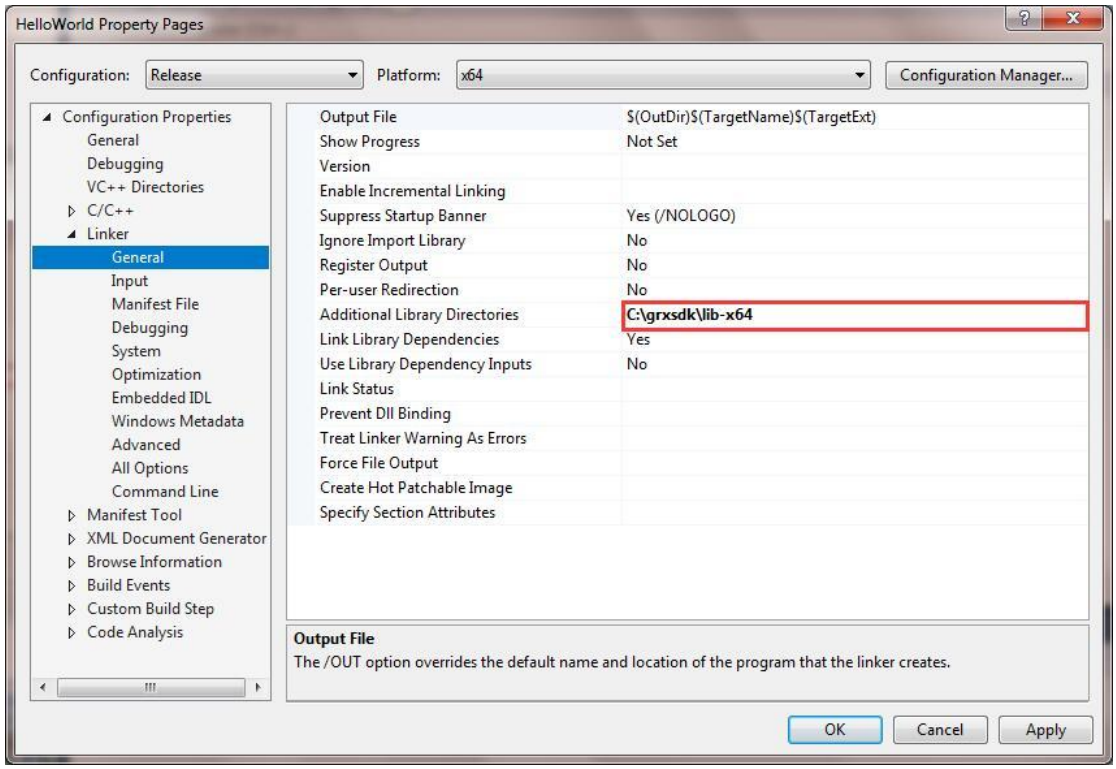
NOTE: If '_DEBUG' needs to be removed from 'DEBUG project configuration'. Please also change 'Runtime Library of Code Generation Node' to 'Multi-threaded Debug DLL (/MDd)' at the same time.

"Language" node：please set compliance mode as "No".

5.4. The Configuration of Linker

1) General Configuration

Select **General** under **linker** of **Configuration Properties as** shown below.

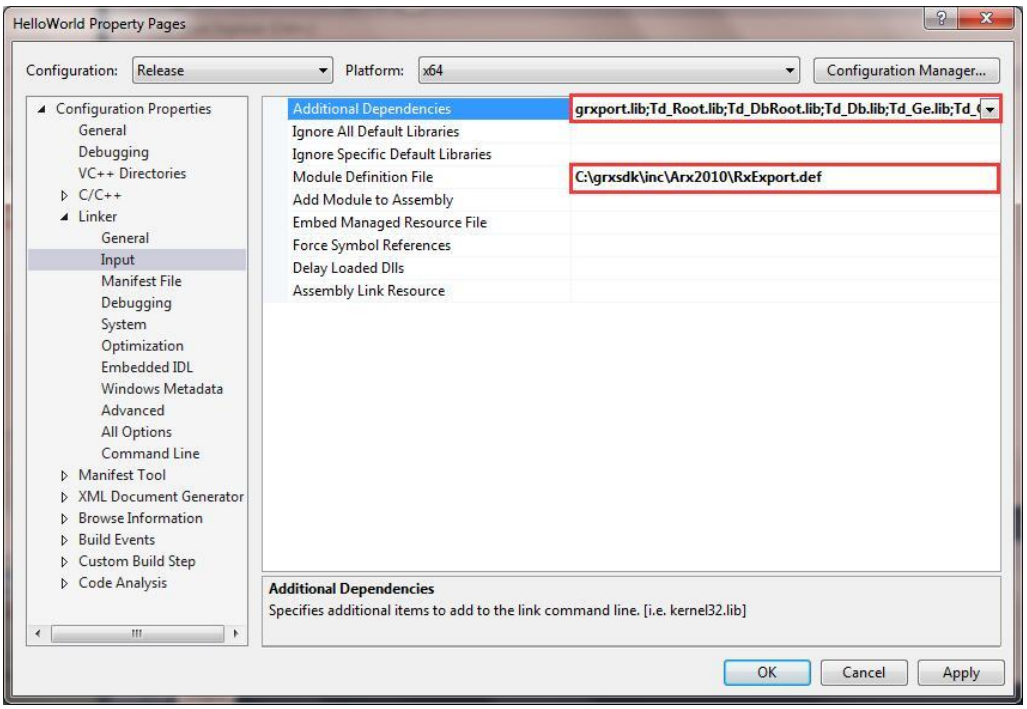


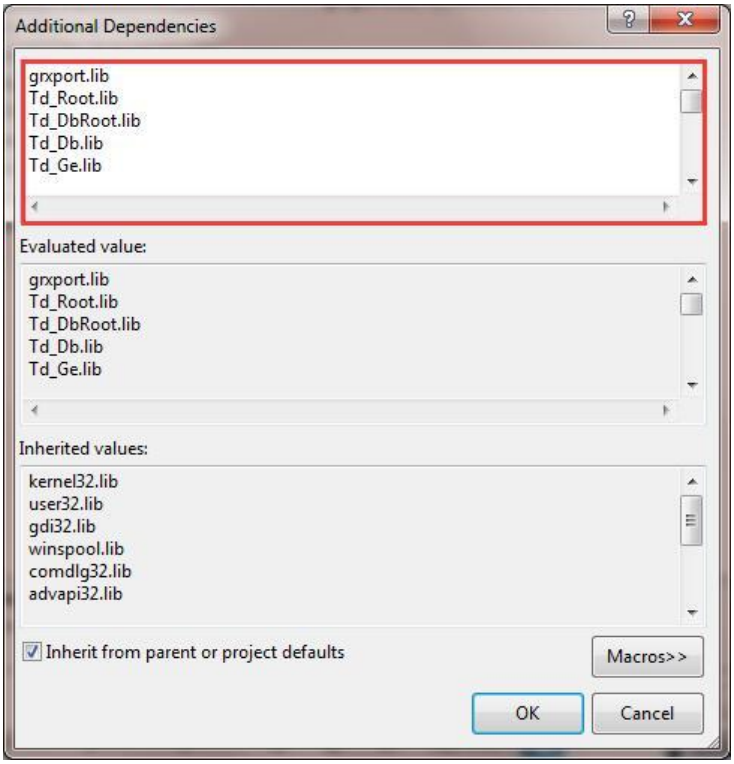
2) Input Configuration

Select Input under Linker of Configuration Properties.

Please add

"grxport.lib,Td_Root.lib,Td_DbRoot.lib,Td_Db.lib,Td_Ge.lib,Td_Gi.lib,Td_Gs.lib,gcad.lib,gcap.lib,gcdb.lib,gced.lib,gcgs.lib,gcut.lib,gcui.lib" (the main GRX library files) to Additional Dependencies. Meanwhile, please set RxExport.def(GRX Enter Point), which is saved in ARX Herder library folders, as Module Definition file. As shown below.





Note: **grxport.lib** must be placed in the first position of additional dependencies.

5.5. Compile Program

After complete above steps. You can compile and run your program.

6. The differences between GRX and ARX interface

6.1. Function: monitorInputPoint

For monitorInputPoint function, the statement between GRX and ARX is different. Use the GCAD_CORE macro to distinguish the different uses of GRX and ARX as shown below.

Example：

```
#ifdef GCAD_CORE
Acad::ErrorStatus AsdkInputPointMonitor::monitorInputPoint(
        bool& appendToTooltipStr,
        ACHAR*& additionalTooltipString,
        AcGiViewportDraw* drawContext,
        AcApDocument* document,
        bool pointComputed,
        int history,
        const AcGePoint3d& lastPoint,
        const AcGePoint3d& rawPoint,
        const AcGePoint3d& grippedPoint,
        const AcGePoint3d& cartesianSnappedPoint,
        const AcGePoint3d& osnappedPoint,
        AcDb::OsnapMask osnapMask,
        const AcArray<AcDbCustomOsnapMode*>& customOsnapModes,
        AcDb::OsnapMask osnapOverrides,
        const AcArray<AcDbCustomOsnapMode*>& customOsnapOverrides,
        const AcArray<AcDbObjectId>& apertureEntities,
        const                                    AcArray<AcDbObjectIdArray,
OdObjectsAllocator<AcDbObjectIdArray > >& nestedApertureEntities,
        const AcArray<Hccad::GsMarker>& gsSelectionMark,
        const AcArray<AcDbObjectId>& keyPointEntities,
        const                                    AcArray<AcDbObjectIdArray,
```

```
OdObjectsAllocator<AcDbObjectIdArray > >& nestedKeyPointEntities,
        const AcArray<Hccad::GsMarker>& keyPointGsSelectionMark,
        const AcArray<AcGeCurve3d*>& alignmentPaths,
        const AcGePoint3d& computedPoint,
        const ACHAR* tooltipString)
#else
virtual Acad::ErrorStatus monitorInputPoint(
        bool& appendToTooltipStr,
        TCHAR*& additionalTooltipString,
        AcGiViewportDraw* drawContext,
        AcApDocument* document,
        bool pointComputed,
        int history,
        const AcGePoint3d& lastPoint,
        const AcGePoint3d& rawPoint,
        const AcGePoint3d& grippedPoint,
        const AcGePoint3d& cartesianSnappedPoint,
        const AcGePoint3d& osnappedPoint,
        AcDb::OsnapMask osnapMasks,
        const AcArray<AcDbCustomOsnapMode*>& customOsnapModes,
        AcDb::OsnapMask osnapOverrides,
        const AcArray<AcDbCustomOsnapMode*>& customOsnapOverrides,
        const AcArray<AcDbObjectId>& apertureEntities,
        const AcArray< AcDbObjectIdArray,
        AcArrayObjectCopyReallocator< AcDbObjectIdArray > >& nestedPickedEntities,
        const AcArray<INT_PTR>& gsSelectionMark,
        const AcArray<AcDbObjectId>& keyPointEntities,
        const AcArray< AcDbObjectIdArray,
        AcArrayObjectCopyReallocator< AcDbObjectIdArray > >&
nestedKeyPointEntities,
```

```
        const AcArray<INT_PTR>& keyPointGsSelectionMark,
        const AcArray<AcGeCurve3d*>& alignmentPaths,
        const AcGePoint3d& computedPoint,
        const TCHAR* tooltipString );
#endif
```

6.2. Function: intersectWith

For intersectWith function, the return vale of intersectWith is different between GRX and ARX when there is no intersection. GRX also return as “eOK” even there is no intersection. Use the GCAD_CORE macro to distinguish the different uses of GRX and ARX as shown below.

Example：

```
bool bHasInters = false;
AcGePoint3dArray aryIntters;
Acad::ErrorStatus es = pCur->intersectWith(pBreak , AcDb::kOnBothOperands , aryIntters);
#ifndef GCAD_CORE
    if ( es != Acad::eOk ) {
    return bHasInters;
    }
#endif
    bHasInters = aryIntters.length() > 0 ;
```

6.3. Function: AcDbField::evaluate

For AcDbField::evaluate function, GRX has to call for the value of the subfield first. Use the GCAD_CORE macro to distinguish the different uses of GRX and ARX as shown below.

Example：

```
AcDbField* pFld = NULL;
if (Acad::eOk == pEnt->getField(_T("TEXT"),pFld,AcDb::kForWrite))
{
#ifdef GCAD_CORE
    // GCAD must call children's evaluate
    int nCount = pFld->childCount();
    for (int i = 0;i<nCount;i++)
    {
        AcDbField* pChiled = NULL;
        if (pFld->getChild(i, pChiled, AcDb::kForWrite) Acad::eOk)
        {
            pChiled->evaluate(AcDbField::kDemand,0);
            pChiled->close();
        }
    }
    pFld->close();
    if (Acad::eOk pEnt->getField(_T("TEXT"),pFld,AcDb::kForWrite))
    {
        if (Acad::eOk == pFld->evaluate(AcDbField::kDemand,0))
        {
            LPTSTR lpstr = NULL ;
            if (Acad::eOk == pFld->getValue(lpstr))
            {
                acutPrintf(L"%s\n", lpstr);
                acutDelString ( lpstr ) ;
                delete lpstr ;
                lpstr = NULL ;
            }
        }
```

```
    }
#else
    if (Acad::eOk == pFld->evaluate(AcDbField::kDemand,0))
    {
        LPTSTR lpstr = NULL ;
        if (Acad::eOk == pFld->getValue( lpstr))
        {
            acutPrintf(L"%s\n", lpstr);
            acutDelString ( lpstr ) ;
            delete lpstr ;
            lpstr = NULL ;
        }
    }
#endif
    pFld->close();
}
```

6.4. Interface for “Plot”

When load Plot interface, the begin Document function in GRX will call beginPage function automatically. Meanwhile, the beginPage will call beginGenerateGraphics function automatically. Which means when you need to load Plot interface, you don’t need to call beginPage, beginGenerateGraphics, endGenerateGraphics and endPage functions any more. Use the GCAD_CORE macro to distinguish the different uses of GRX and ARX as shown below.

Example：

```
AcPlPlotEngine* mPlotEngine;
es = AcPlPlotFactory::createPublishEngine(mPlotEngine);
es = mPlotEngine->beginPlot(NULL);
```

```
    AcPlPlotInfo plotInfo;
    AcPlPlotInfoValidator plotInfoValidator;
    plotInfo.setLayout(mPlotSettings->id());
    plotInfo.setOverrideSettings(mPlotSettings);
    plotInfoValidator.setMediaMatchingPolicy(AcPlPlotInfoValidator::kMatchEnabled);
    es = plotInfoValidator.validate(plotInfo);
    es = mPlotEngine->beginDocument(plotInfo, curDoc()->fileName(), NULL,1, true,
_T("c:\\test.eps"));
#ifndef GCAD_CORE
    AcPlPlotPageInfo pageInfo;
    es = mPlotEngine->beginPage(pageInfo, plotInfo, true);
    es = mPlotEngine->beginGenerateGraphics();
    es = mPlotEngine->endGenerateGraphics();
    es = mPlotEngine->endPage();
#endif
    es = mPlotEngine->endDocument();
    es = mPlotEngine->endPlot();
    mPlotEngine->destroy();
```

6.5. Gs initialization

The difference for Gs initialization, please refer to the GCAD_CORE macro to distinguish the different uses of GRX and ARX as shown below.

Example：

```
    Acad::ErrorStatus initDrawingCtrl(AcDbDatabase *pDb,const TCHAR *chSpace)
    {
        // have we got a valid drawing
        if (NULL == pDb)
    {
```

```
            return Acad::eNullBlockName;
        }
        // initialize the preview control
        mPreviewCtrl.init();
        mPreviewCtrl.SetFocus();
        AcDbBlockTableRecordPointer bptr(chSpace, pDb, AcDb::kForRead);
        AcDbObjectId idCurrenVs;
        // set the view to the Active AutoCAD view
        idCurrenVs = SetViewTo(mPreviewCtrl.mpView, pDb, m_viewMatrix);
        // tell the view to display this space
        mPreviewCtrl.view()->add(bptr, mPreviewCtrl.model());
        mPreviewCtrl.view()->setVisualStyle(idCurrenVs);
        return Acad::eOk;
    }
#ifdef GCAD_CORE
    AcGePoint3d ptMin ( 1e20, 1e20, 1e20);
    AcGePoint3d ptMax ( -1e20, -1e20, -1e20);
    m_tempExt.set ( ptMin , ptMax ) ;
    OdDbDatabasePtr pDb = acdbHostApplicationServices()->createDatabase();
    initDrawingCtrl(pDb,L"*Model_Space");
#else
    m_tempExt.minPoint().set(1e20, 1e20, 1e20);
    m_tempExt.maxPoint().set(-1e20, -1e20, -1e20);
    AcDbDatabase* pDb = new AcDbDatabase;
    initDrawingCtrl(pDb,L"*Model_Space");
#endif
```

6.6. Gs Pan

When you need to call Gs Pan function, it's different between GRX and ARX. You have to use it upside down. Use the GCAD_CORE macro to distinguish the different uses of GRX and ARX as shown below.

Example：

```
#ifdef GCAD_CORE
    AcGeVector3d pan_vec( -(point.x-mStartPt.x), -(point.y-mStartPt.y), 0 );
#else
    AcGeVector3d pan_vec( -(point.x-mStartPt.x), point.y-mStartPt.y, 0 );
#endif
    pan_vec.transformBy(mpView->viewingMatrix() *
mpView->worldToDeviceMatrix().inverse());
    mpView->dolly(pan_vec);
    Invalidate();
mStartPt = point;
```

6.7. acedGetArgs() release issue

The resbuf returned by acedGetArgs () cannot be released. (No need to call acutRelRb ()).

7. Comparison of GRX and ARX libraries

7.1. GRX class library description

The following libraries are frequently used in GRX. These libraries have the similar effect with the corresponding libraries under ARX, mainly including:：

- GcRx- has the similar effect with the AcRX library, which is used to bind an application and register and identify for the running class.。
- GcEd- has the similar effect with the AcEd library, used to register custom commands and event notifications.
- GcDb- has the similar effect with AcDb library, it's GstarCAD database class.
- GcGi- has the similar effect with the AcGi library and is used for the graphics class of GstarCAD.
- GcGe- has the similar effect with the AcGe library, and is used for general linear algebra calculation and application classes of geometric objects.

7.1.1. GcRx

The GcRx class library is used for DLL initialization and the system level for running class registration and identification. It provides the functions below:

- Class identification and inheritance analysis of object runtime.
- Add a new protocol to an existing class at runtime.
- Object equality and composition judgment.
- Object copy.

7.1.2. GcEd

The GcEd class library is used to define and register new GstarCAD commands, which are the same as GstarCAD internal/original commands.

7.1.3. GcDb

The GcDb class library represents the classes that build up the GstarCAD database. The database stores the information of all graphic objects. These graphic objects are called “Entities”. These graphic objects and non-graphic entities (such as layers, line types, and text styles) constitute GstarCAD graphics.

7.1.4. GcGi

GcGe library provides the graphic interface which is used to draw CAD entities.

7.1.5. GcGe

GcGe library provides some geometric tools (e.g., vector and matrix) to perform the 2D and 2D geometric operations, it also provides the basic geometric objects, such as point, curve and curved surface.

8. Copyright

Copyright reserved: Xi'an Gstarsoft Co.,Ltd

Usage License: Allows copying, quote any part of this document. Changes of any part of this document are forbidden without authorization. Please keep this statement when copying and quoting it, otherwise it will be held liable.

* AutoCAD® is a product of Autodesk® Company which is one of the CAD software solution providers. ARX is an abbreviated name of ObjectARX, which is AutoCAD® Runtime eXtension, and C++ SDK provided by Autodesk®